M2. Encapsulated Mess-Card Wallet

Week 3 OOP | Category B | Java solution with sample output

Objective

Implement the task using the classes, constructors, access control, references, or static members specified in the uploaded problem sheet.

Java Code

```
class MessWallet {
    private double balance;

    public MessWallet(double openingBalance) {
        if (openingBalance < 0) {
            System.out.println("Warning: negative opening balance replaced with 0.");
            balance = 0;
        } else {
            balance = openingBalance;
        }
    }

    public void topUp(double amount) {
        if (amount <= 0) {
            System.out.println("Top-up rejected: amount must be positive.");
            return;
        }
        balance += amount;
    }

    public void deduct(double amount) {
        if (amount <= 0) {
            System.out.println("Deduct rejected: amount must be positive.");
        } else if (amount > balance) {
            System.out.println("Deduct rejected: insufficient balance");
        } else {
            balance -= amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        MessWallet wallet = new MessWallet(500);
        wallet.topUp(200);
        System.out.println("Balance after top-up: " + wallet.getBalance());
        wallet.deduct(1000);
        System.out.println("Final balance: " + wallet.getBalance());
    }
}
```

Sample Output

```
Balance after top-up: 700.0
Deduct rejected: insufficient balance
Final balance: 700.0
```

Concepts Covered

Encapsulation, private balance, validation, read-only getter